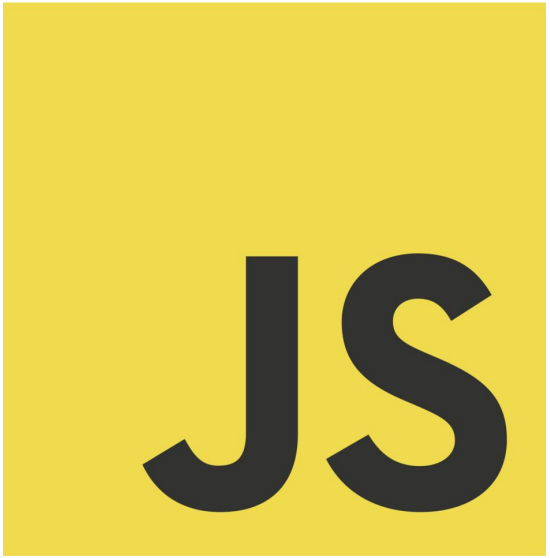
Atualizado em 08.10.2026
Versão 0.0.1

JavaScript

Programação assíncrona
Promises / Async functions



Fetch © 2026 por Allyson Souza está licenciado sob a **CC BY-NC 4.0**.
Para ver uma cópia da licença, visite
https://creativecommons.org/licenses/by-nc/4.0/

Promises

Promises

O objeto **Promise** representa a eventual conclusão (ou falha) de uma operação assíncrona e seu valor resultante.

Essencialmente, uma promise é um objeto retornado para o qual você adiciona *callbacks*, em vez de passar *callbacks* para uma função.

Estados de um Promise

pending: estado inicial, nem cumprido nem rejeitado.

fulfilled: significa que a operação foi concluída com sucesso.

rejected: significa que a operação falhou.

Sem Promise...

Por exemplo, em vez de uma função convencional que recebe dois *callbacks*, e invoca um deles em uma eventual conclusão ou falha, como a seguir:

```
function successCallback(result) {
  console.log("It succeeded with " + result);
}

function failureCallback(error) {
  console.log("It failed with " + error);
}

doSomething(successCallback, failureCallback);
```

Com Promise

…funções modernas retornam uma promise e então você pode adicionar seus callbacks:

Nós chamamos isso de chamada de **função assíncrona**.

```
const promise = doSomething();

promise.then(successCallback, failureCallback);

// Ou então, apenas...
doSomething().then(successCallback, failureCallback);
```

Garantias

Ao contrário dos *callbacks* com funções como parâmetro, uma promise vem com algumas garantias:

- *Callbacks* nunca serão chamados antes da conclusão da execução atual do loop de eventos do JavaScript
- *Callbacks* adicionadas com **.then,** mesmo depois do sucesso ou falha da operação assíncrona, serão chamadas
- Múltiplos callbacks podem ser adicionados chamando-se **.then** várias vezes, para serem executados independentemente da ordem de inserção

Mas o benefício mais imediato das promises é o encadeamento.

Encadeamento

Uma necessidade comum é executar duas ou mais operações assíncronas consecutivas, onde cada operação subsequente começa quando a operação anterior é bem sucedida, com o resultado do passo anterior. Nós conseguimos isso criando uma cadeia de promises.

A função **then** retorna uma nova **promise**, diferente da original.

Encadeamento

```
const promise = doSomething();
const promise2 = promise.then(successCallback, failureCallback);

// OU
const promise2 = doSomething().then(successCallback, failureCallback);
```

"Pirâmide da desgraça"

```
facaAlgo(function (resulto) {

  facaAlgoMais(
    resulto,
    function (novoResultado) {
      umaTerceiraCoisinha(
        novoResultado,
        function (resultadoFinal) {
          console.log("Finalmente, o resultado: " + resultadoFinal);
        },
        callbackCasoFalha,
      );
    },
    callbackCasoFalha,
  );
}, callbackCasoFalha);
```

Código com promises

```
facaAlgo()
  .then(function (resultado) {
    return facaAlgoMais(resultado);
  })
  .then(function (novoResultado) {
    return umaTerceiraCoisinha(novoResultado);
  })
  .then(function (resultadoFinal) {
    console.log("Finalmente, o resultado: " + resultadoFinal);
  })
  .catch(callbackCasoFalha);


// Os argumentos para then são opcionais
// catch(failureCallback) é uma abreviação para then(null, failureCallback)
```

Código com promises e arrow functions

```
facaAlgo()
  .then((resultado) => facaAlgoMais(resultado))
  .then((novoResultado) => umaTerceiraCoisinha(novoResultado))
  .then((resultadoFinal) => {
    console.log(`Finalmente, o resultado: ${resultadoFinal}`);
  })
  .catch(failureCallback);
```

// Importante: sempre retorne um resultado, de outra forma as *callbacks* não vão capturar o resultado da promise anterior.

Encadeamento depois de falha

É possível encadear depois de uma falha (um ***catch***).

Isso é muito útil para realizar novas ações mesmo depois que uma falha ocorra no encadeamento.

Encadeamento depois da falha

```
new Promise((resolve, reject) => {
  console.log("Início");
  resolve();
})
  .then(() => {
    throw new Error("Algo deu errado!");
    console.log("Callback de sucesso não executado");
  })
  .catch(() => {
    console.log("Callback de erro executado");
  })
  .then(() => {
    console.log("Callback executado após erro");
  });
```

Propagação de erros

Uma corrente de promises para se houver uma exceção, procurando por ***catch handlers*** ao invés de seguir seu encadeamento esperado.

Essa modelagem de código segue bastante a maneira de como o código síncrono funciona.

Captura de erros em código assíncrono

```
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => console.log(`Got the final result: ${finalResult}`))
  .catch(failureCallback);
```

Captura de erros em código síncrono

```
try {
  const result = syncDoSomething();
  const newResult = syncDoSomethingElse(result);
  const finalResult = syncDoThirdThing(newResult);
  console.log(`Got the final result: ${finalResult}`);
} catch (error) {
  failureCallback(error);
}
```

Composição

...

Composição de operações assíncronas

Existem quatro ferramentas de composição para executar operações assíncronas de forma concorrente:

- `Promise.all()`
- `Promise.allSettled()`
- `Promise.any()`
- `Promise.race()`

Promise.all()

```
Promise.all([func1(), func2(), func3()])
    .then(([result1, result2, result3]) => {
       // use result1, result2 and result3
    });
```

Criando uma Promise

...

Exemplo de função assíncrona

```
// Criando nova Promise
const buscarDados = new Promise((resolve, reject) => {
  // Simulando uma condição
  let sucesso = true;
  setTimeout(() => {
    if (sucesso) {
      resolve({ id: 1, nome: "Maria" });
    } else {
      reject("Erro ao carregar os dados!");
    }
  }, 2000);
});
```

```
// Consumindo a Promise com .then() e .catch()
buscarDados
  .then((dados) => {
    console.log("Sucesso:", dados);
  })
  .catch((erro) => {
    console.error("Erro:", erro);
  });
```

Async function

...

Async function

A declaração **async function** define uma função assíncrona, que retorna um objeto **AsyncFunction**.

A palavra-chave **await** é permitida dentro do corpo da função, permitindo que um comportamento assíncrono — baseado em promises — seja escrito de uma forma mais limpa, eliminando a necessidade de configurar explicitamente cadeias de promises.

Detalhes sobre async functions

Quando uma função assíncrona é chamada, ela retorna uma **Promise**. Quando a função assíncrona retorna um valor, a Promise será **resolvida** com o valor retornado. Quando a função assíncrona lança uma exceção ou algum valor, a Promise será **rejeitada** com o valor lançado.

Uma função assíncrona pode conter uma expressão await, que pausa a execução da função assíncrona e espera pela resolução da Promise passada, e depois retoma a execução da função assíncrona e retorna o valor resolvido.

Exemplo de função assíncrona

```
// Função simulada que retorna uma Promise
function buscarDados() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve({ id: 1, nome: "Maria" });
    }, 2000);
  });
}
```

```
// Função assíncrona usando async/await
async function executar() {
  try {
    console.log("Buscando dados...");
    // Pausa aqui até a Promise resolver
    const dados = await buscarDados();

    console.log("Sucesso:", dados);
  } catch (erro) {
    console.error("Erro capturado:", erro);
  }
}
executar();
```

Referências Bibliográficas

MDN - Mozilla Developer Network. **Using promises**. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises.